npm との違いを手がかりに Go の依存管理を理解したい

白木(shirokuma)

Go Bash vol.3

登壇日：2026/09/30

© 2019-2026 OPTiM Corp. All rights reserved.

自己紹介

- 白木 (shirokuma)
- 4年目
- 趣味
  - ■読書📚
  - ■テニス🎾
  - ■Designship 2026 の運営
- プロダクト
  - ■OPTiM AIR 2023年 8月~
    - ●自社のPaaS基盤の開発と保守
  - ■OPTiM Collaboration Portal 2025年 6月~
    - ●ポータルサイト作成・管理のアプリの開発
  - ■OPTiM Biz AI Agent 2026年 7月~
    - ●生成AI活用基盤アプリの開発

OPTiM

OPTiM AIR

OPTiM Collaboration Portal

OPTiM Biz AI Agent (β)



© 2019-2026 OPTiM Corp. All rights reserved.

会社紹介

© 2019-2026 OPTiM Corp. All rights reserved.

基本情報

商号	株式会社オプティム（プライム市場：3694） Optimization:最適化 × Optimism:楽観主義
設立	2000年
オフィス	OPTiM TOKYO（東京本社@浜松町） OPTiM SAGA（佐賀本店@佐賀大学キャンパス内） OPTiM KOBE （神戸オフィス@三ノ宮） TECH CENTER IIZUKA （テックセンター飯塚@九州工業大学飯塚キャンパス前）
代表者	菅谷 俊二
社員数	444名（2026年4月現在） うち約７割がエンジニア職



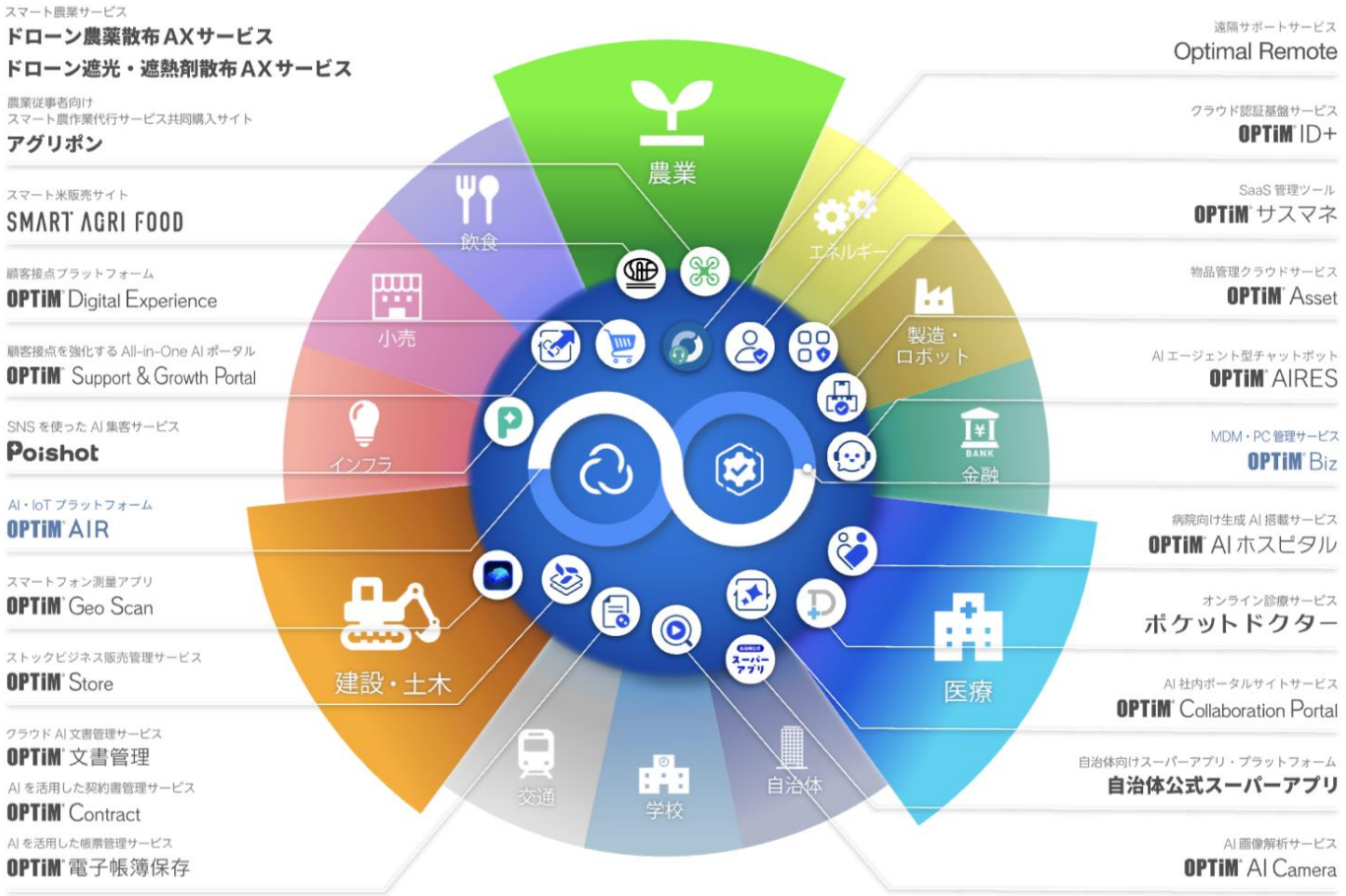




© 2019-2026 OPTiM Corp. All rights reserved.

事業・提供サービス概要

私たちは、AI・IoT技術を社会の隅々に行き渡らせ、
日本のあらゆる産業を根底から変革する
「AXの中核を担う企業」を目指しています。



© 2019-2026 OPTiM Corp. All rights reserved.

本題

© 2019-2026 OPTiM Corp. All rights reserved.

npm と Go の一番の違いは何だと思いますか？

© 2019-2026 OPTiM Corp. All rights reserved.

Go と npm の比較

比較対象	Go	npm
バージョンを付けて配布・取得される単位	module	package
コードを読み込む単位	package（同じディレクトリにあり、まとめてコンパイルされる .go ファイル群）	module（require / import で読み込めるファイルやディレクトリ）
packageの場所	GOMODCACHE マシンで1つ	プロジェクトごとに node_modules 配下に展開
同一バージョンの重複	無い	プロジェクト数だけコピー
バージョンの宣言	go.mod にピンポイント（最低版）	package.json に範囲 + lock に結果
書き込み	読み取り専用	書き換え可能
git 管理	go.mod と go.sum を管理 GOMODCACHE は管理外	package.json と package-lock.json を管理 node_modules は管理外(.gitignore 推奨)

© 2019-2026 OPTiM Corp. All rights reserved.

依存ライブラリが置かれる場所の違い

npm（Express）

```
# カレントディレクトリ
$ pwd
/Users/opm008296/git/github.com-takumashiraki/takumashiraki/conference-report/GoBash/ex-npm

# express の絶対 path を表示
$ npm ls express --parseable
/Users/opm008296/git/github.com-takumashiraki/takumashiraki/conference-report/GoBash/ex-npm/node_modules/express
```

Go（chi）

```
# キャッシュとして保存されるディレクトリのパス
$ go env GOMODCACHE
/Users/opm008296/go/pkg/mod

# ダウンロードしたバージョンのパッケージの path を表示
$ go list -m -f '{{.Dir}}' github.com/go-chi/chi/v5
/Users/opm008296/go/pkg/mod/github.com/go-chi/chi/v5@v5.3.2
```

```
【Go】
 プロジェクトA ┐
               ├→  ~/go/pkg/mod/github.com/go-chi/chi/v5@v5.3.2
 プロジェクトB ┘      マシンに1つ・読み取り専用

【npm】
 プロジェクトA/node_modules/express    ← それぞれ別のコピー
 プロジェクトB/node_modules/express
```

© 2019-2026 OPTiM Corp. All rights reserved.

バージョンの宣言の違い

npm（package.json, package-lock.json ）

```
{
  // 5.2.1 以上 6.0.0 未満を使う
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

バージョンの
範囲を指定

```
{
  // 5.2.1 を入れた
  "node_modules/express": {
    "version": "5.2.1",
  },
  // Node 18 以上を対象とする
  "engines": {
    "node": ">= 18"
  },
}
```

バージョンを記録

Go（go.mod）

```
// https://go.dev/ref/mod#go-mod-file-module
// module ディレクティブは、そのモジュール自身のモジュールパスを定義する
module github.com/takumashiraki/conference-report/GoBash/ex-Go

// https://go.dev/ref/mod#go-mod-file-go
// go ディレクティブは、そのモジュールがどの Go バージョンの仕様・挙動を
前提として書かれているかを示す
go 1.26.8

// https://go.dev/ref/mod#go-mod-file-require
// require ディレクティブは、依存するモジュールについて必要となる最低バー
ジョンを指定する
require github.com/go-chi/chi/v5 v5.3.2
```

↓モジュールパス

最低バージョンを宣言

© 2019-2026 OPTiM Corp. All rights reserved.

npm との比較で感じた疑問

- なぜ同じディレクトリに配置しているのか？
- 別のプロジェクトが別のバージョンを使いたくなったら？
- 別のプロジェクトが中身を書き換えたら？
- バージョンタグを別のコミットに付け替えたら？

© 2019-2026 OPTiM Corp. All rights reserved.

OPTiM

go get の仕組みを理解し、疑問の答えを探る

Go 1.26.8 を対象とする

© 2019-2026 OPTiM Corp. All rights reserved.

go get の処理に答えがありそう

OPTiM

1. 必要なモジュールとバージョンを決める
2. モジュールプロキシ（プロキシサーバー）または VCS （Gitなど）からモジュールを取得する
3. モジュールキャッシュに保存する
4. 暗号学的ハッシュを計算する
5. go.sum または checksum database（sum.golang.org）と照合する
6. go.sum に記録する

```
$ go get github.com/go-chi/chi/v5
go: downloading github.com/go-chi/chi/v5 v5.3.2 ← STEP1
go: downloading github.com/go-chi/chi v1.5.5    ← STEP1
go: added github.com/go-chi/chi/v5 v5.3.2        ← STEP6

$ cat go.sum                                     ↓ STEP6(記録結果確認)
github.com/go-chi/chi/v5 v5.3.2 h1:5YQkICvTCSZ25hoRsyJazN0scjzKGiu4VAUc7H1o1nY=
github.com/go-chi/chi/v5 v5.3.2/go.mod h1:R+tYY2hNuVUUjxoPtqUdgBqevM9s9njzkTLutVsOCto=
```

© 2019-2026 OPTiM Corp. All rights reserved.

STEP1 必要なモジュールとバージョンを決める

OPTiM

1. import path からモジュールパス候補を列挙

1. ビルドリスト照合する
   - ビルドリストから見つからない場合はモジュールプロキシに問い合わせる
2. モジュールプロキシに問い合わせる
   - モジュールプロキシから必要なモジュールとバージョンの候補を取得する

2. MVS（Minimal Version Selection）

1. モジュールグラフをたどってバージョンを決める

```
package main

import (
    "fmt"
    "log"
    "net/http"

    "github.com/go-chi/chi/v5"
)
```

import path

```
go list -m all
github.com/takumashiraki/conference-
report/GoBash/ex-Go
github.com/go-chi/chi/v5 v5.3.2
```

ビルドリスト

用語

- **ビルドリスト**
  - ビルドに使うモジュールとバージョンの一覧。go list -m all で確認できる
- **import path**
  - モジュールパスとモジュール内のサブディレクトリをつないだもの
- **モジュールグラフ**
  - メインモジュールを起点に、各モジュールの go.mod の require をたどってできる依存関係の図

© 2019-2026 OPTiM Corp. All rights reserved.

STEP1.1 import path からモジュールパス候補を列挙(ビルドリスト照合)

OPTiM

- github.com/go-chi/chi/v5 という import path を見ても、 Go コマンドは「どこまでがモジュール名で、どこからがパッケージのサブディレクトリか」を知らない
- そのため、 Go コマンドはビルドリストの中から、パスが import path の接頭辞になっているモジュールを探す
- 見つからない場合 go get と go mod tidy は**新しいモジュールを探す処理に進む**

```go
package main

import (
    "fmt"
    "log"
    "net/http"

    "github.com/go-chi/chi/v5"
)
```

import path

```
$ go list -m all
# ↓ メインモジュール
github.com/takumashiraki/conference-report/GoBash/ex-Go
# ↓ 依存モジュール（モジュールパス バージョン）
github.com/go-chi/chi/v5 v5.3.2
```

見つかった場合

© 2019-2026 OPTiM Corp. All rights reserved.

STEP1.1 import path からモジュールパス候補を列挙（プロキシで存在確認）

OPTiM

- **import path の末尾から 1 要素ずつ削っていき、得られた各パスを、そのパッケージを提供しうるモジュールパスの候補とする**
  - ■GOPROXY の各エントリに対して、候補ごとに最新バージョンを要求する
  - ■公式リファレンスの例では、これらの要求は 1 つのプロキシに対して並列に送られる
- **要求に成功したモジュールパスについては、最新バージョンのモジュールを取得し、要求されたパッケージを含むかどうかを確認する**
- **パッケージを含むモジュールが複数あれば、パスが最も長いモジュールを使う**

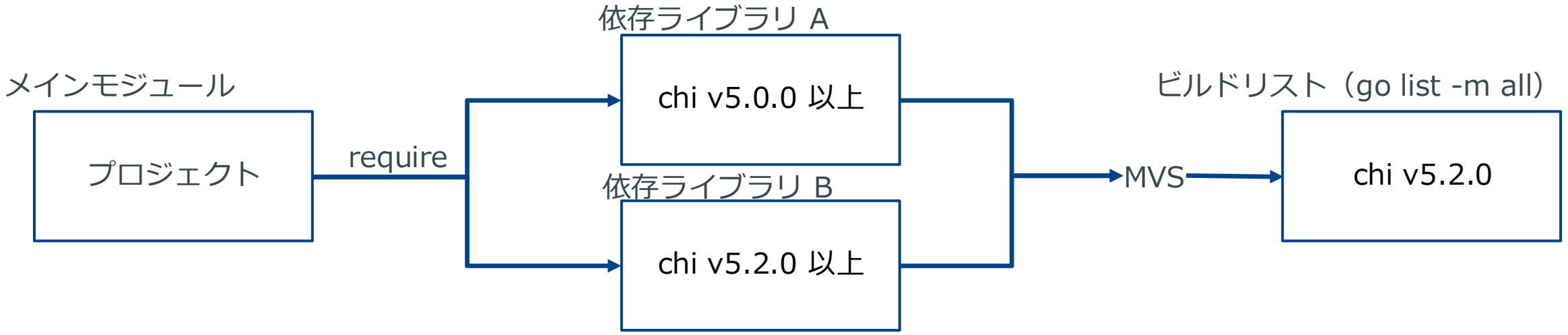
```
$ go get -x github.com/go-chi/chi/v5 2>&1 | grep -E '/@v/list: |go: downloading'
# get https://proxy.golang.org/github.com/go-chi/chi/v5/@v/list: 200 OK
# get https://proxy.golang.org/github.com/go-chi/chi/@v/list: 200 OK
# get https://proxy.golang.org/github.com/go-chi/@v/list: 404 Not Found
# get https://proxy.golang.org/github.com/@v/list: 404 Not Found
go: downloading github.com/go-chi/chi/v5 v5.3.2
go: downloading github.com/go-chi/chi v1.5.5
```

```
$ go get github.com/go-chi/chi/v5
go: downloading github.com/go-chi/chi/v5 v5.3.2
go: downloading github.com/go-chi/chi v1.5.5
go: added github.com/go-chi/chi/v5 v5.3.2
```

© 2019-2026 OPTiM Corp. All rights reserved.

STEP1.2 MVS（Minimal Version Selection）

- 候補が確定したら、モジュールグラフをたどってバージョンを決める
- MVS はメインモジュールからグラフをたどり、各モジュールについて要求された中で**最も高い**バージョンを記録する
- たどり終えた時点で記録されている**最も高いバージョンの集合がビルドリストになる**
- すべての要求を満たす最小のバージョンを使用する
  - ■「最も高い」と「最小」は矛盾しない
  - ■A が chi v5.0.0 以上、B が v5.2.0 以上を要求していれば、両方を満たす最小のバージョンは v5.2.0



© 2019-2026 OPTiM Corp. All rights reserved.

STEP2 モジュールプロキシまたは VCS から取得する

OPTiM

取得経路は 2 通り

- ■モジュールプロキシ経由
- ■direct 経由 (今回説明は省く)

```
$ go env GOPROXY
https://proxy.golang.org,direct
```

場面	経路	例
公開モジュール	モジュールプロキシ	github.com/go-chi/chi/v5
プライベートモジュール	direct	弊社の場合 gitlab.tokyo.optim.co.jp/*
公開モジュールだが、モジュールプロキシが配信拒否した場合	direct (フォールバック)	主に法的な理由のとき

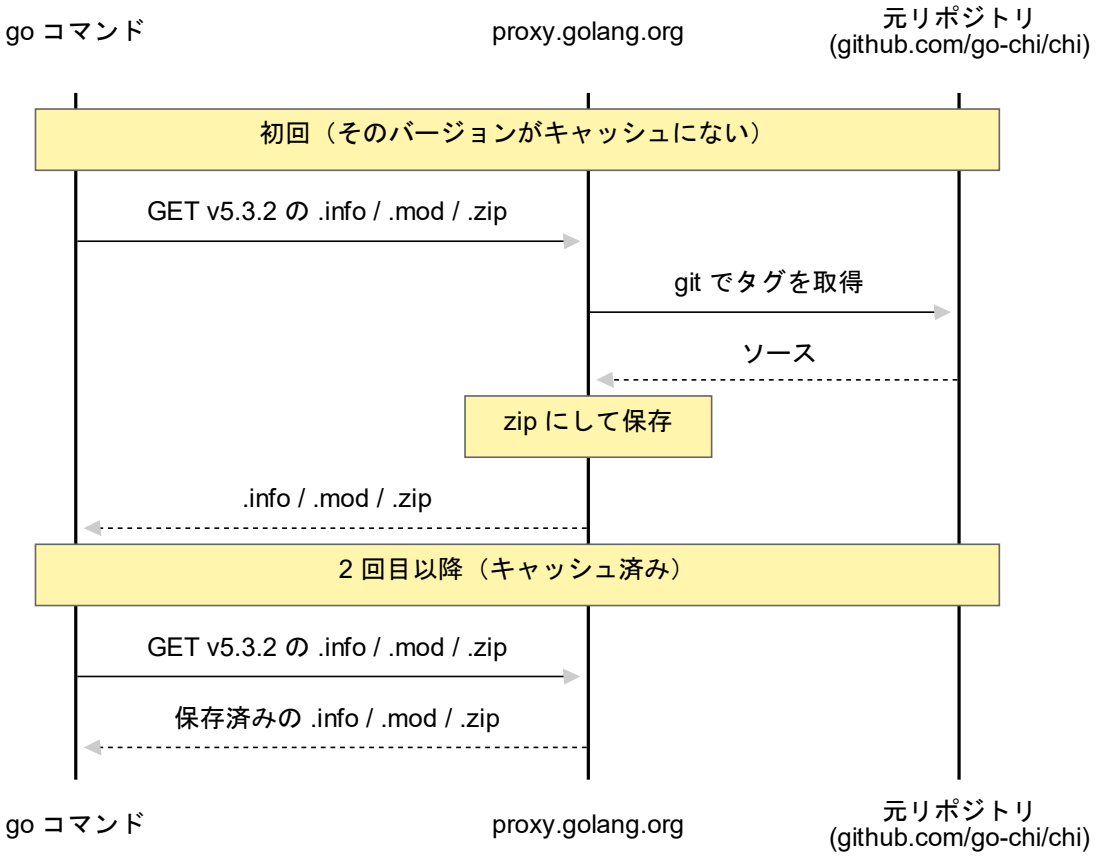
用語

- **https://proxy.golang.org**
  - ■モジュールプロキシと呼ぶ(モジュールのキャッシュサーバーのこと)
- **,**
  - ■フォールバックのチェーン (「1 番目で取れなければ 2 番目を試す」という順番付きの候補リストで、モジュールプロキシが 404 / 410 を返したら次に進む)
- **direct**
  - ■プロキシを使わず VCS から直接取得する

© 2019-2026 OPTiM Corp. All rights reserved.

STEP2.1 モジュールプロキシ経由の場合(モジュールプロキシの役割)

- GitHub などから取得したソースを zip 化して保存しておき、 Go コマンドに HTTP で渡す
- npm の場合は開発者が npm registry (パッケージを配布するサーバー)にアップロードする
- Go の場合はモジュールプロキシ自身が元リポジトリから取得する
- いったんキャッシュされると、開発者が元リポジトリでソースを消しても取得できる



© 2019-2026 OPTiM Corp. All rights reserved.

STEP2.1 モジュールプロキシ経由の場合(バージョン一覧の取得手順)

OPTiM

- バージョン無指定の go get の場合 @v/list でバージョン一覧を取る
- その中からプレリリースを除き、SemVer（Semantic Versioning）で最大のバージョンを選び .info → .mod → .zip の順にリクエストする
  - .mod と .zip の間には checksum database へのハッシュ照合が挟まる

```
$ GOMODCACHE=$(mktemp -d) GOFLAGS=-modcacherw go get -x github.com/go-chi/chi/v5 2>&1
# get https://proxy.golang.org/github.com/go-chi/chi/v5/@v/list 200 OK        ← @v/list でバージョン一覧取得
# get https://proxy.golang.org/github.com/go-chi/chi/v5/@v/v5.3.2.info 200 OK ← 最大のバージョンのメタデータ
# get https://proxy.golang.org/github.com/go-chi/chi/v5/@v/v5.3.2.mod 200 OK  ← v5.3.2 のgo.mod を取得
# get https://sum.golang.org/lookup/github.com/go-chi/chi/v5@v5.3.2 200 OK    ← 照合用ハッシュを取得
go: downloading github.com/go-chi/chi/v5 v5.3.2
# get https://proxy.golang.org/github.com/go-chi/chi/v5/@v/v5.3.2.zip 200 OK  ← モジュール zip ファイル
go: added github.com/go-chi/chi/v5 v5.3.2
```

```
$ curl -s https://proxy.golang.org/github.com/go-chi/chi/v5/@v/list | sort -V | tail -3
v5.3.0
v5.3.1
v5.3.2
```

© 2019-2026 OPTiM Corp. All rights reserved.

STEP3 モジュールキャッシュに保存する

OPTiM

- 保存先は GOMODCACHE（デフォルトは ~/go/pkg/mod ）である

1. モジュールプロキシ（または VCS ）から取得した .info・ .mod ・ .zip を層1に保存する
2. モジュール zip ファイルを層2に展開し、読み取り専用にする
3. 2回目以降の go build / go test は層2のソースだけを読む（zip は再展開しない）

```
~/go/pkg/mod/
├── cache/                          ← 層1: 取得したものをそのまま置く
│   ├── download/
│   │   ├── github.com/go-chi/chi/v5/@v/
│   │   │   ├── list                ←   手元にあるバージョンの一覧
│   │   │   ├── v5.3.2.info         ←   バージョンと取得元（git のコミット）
│   │   │   ├── v5.3.2.mod          ←   go.mod 単体
│   │   │   ├── v5.3.2.zip          ←   ソースアーカイブ（ソース一式の zip）
│   │   │   ├── v5.3.2.ziphash      ←   zip のハッシュ（go.sum と同じ値）
│   │   │   └── v5.3.2.lock
│   │   └── sumdb/                  ←   チェックサム DB の応答（ステップ5）
│   └── vcs/                        ←   direct 取得時の git リポジトリ（ステップ2）
│
└── github.com/go-chi/chi/          ← 層2: zip を展開したソース（ビルドが読む）
    ├── v5@v5.2.5/
    ├── v5@v5.3.1/
    └── v5@v5.3.2/                  ←   chi.go, mux.go, go.mod ...
```

```
$ go get github.com/go-chi/chi/v5
go: downloading github.com/go-chi/chi/v5 v5.3.2
go: downloading github.com/go-chi/chi v1.5.5
go: added github.com/go-chi/chi/v5 v5.3.2
```

© 2019-2026 OPTiM Corp. All rights reserved.

STEP4 暗号学的ハッシュを計算する

OPTiM

1. **モジュール zip ファイル内の全ファイルに対して SHA-256 を計算する**
   1. STEP3 で層1に保存した v5.3.2.zip の README や LICENSE も含めて全てを計算する
2. **"< SHA-256 の hex> <ファイル名>¥n" という行を作る（スペースは 2 個）**
3. **ファイル名でソートして全行を連結する → これが「リスト」**
4. **そのリスト全体の SHA-256 を取り、その 32 バイトを base64 化して "h1:" を付ける**
   - h1: は SHA-256 を使うハッシュ方式「Hash1」の識別子

```
$ cd $(mktemp -d)
$ unzip -q $(go env GOMODCACHE)/cache/download/github.com/go-chi/chi/v5/@v/v5.3.2.zip
$ find github.com -type f | LC_ALL=C sort |  # STEP3: ファイル名でソート
    xargs shasum -a 256 |                    # STEP1〜3: 各ファイルの SHA-256 を「<hex>  <ファイル名>」の行にして連結
    shasum -a 256 | cut -d' ' -f1 |          # STEP4: リスト全体の SHA-256
    xxd -r -p | base64                       # STEP4: hex を 32 バイトに戻して base64
5YQkICvTCSZ25hoRsyJazN0scjzKGiu4VAUc7H1o1nY=
# 先頭に h1: を付けると go.sum の 1 行目と一致
```

← Go コマンドは dirhash で計算してる

© 2019-2026 OPTiM Corp. All rights reserved.

STEP5 go.sum または checksum database と照合する

OPTiM

1. go.sum にすでに行がある場合

■ハッシュが一致する時は checksum database に問い合わせずに検証を終えます

■ハッシュが一致しない時は取得結果を捨ててエラーにします

2. go.sum に無い場合

■checksum database （デフォルトは “sum.golang.org” ）からモジュールのハッシュを受け取ります

■Go コマンドに組み込まれた公開鍵で署名を検証してから、ダウンロードしたモジュールと照合します

```
$ curl https://sum.golang.org/lookup/github.com/go-chi/chi/v5@v5.3.2
60295460 # ← sumdb のログ上でこのレコードが何番目か
github.com/go-chi/chi/v5 v5.3.2 h1:5YQkICvTCSZ25hoRsyJazN0scjzKGiu4VAUc7H1o1nY= # ← zip 内の全ファイルのハッシュ
github.com/go-chi/chi/v5 v5.3.2/go.mod h1:R+tYY2hNuVUUjxoPtqUdgBqevM9s9njzkTLutVsOCto= # ← go.mod のハッシュ

go.sum database tree # ← ここから下は署名付きツリーヘッド
63096908 # ← 応答時点でのツリーのサイズ
8lHrQPRYe4KT4WQ9/Qk8/taKnD6IRHbFgfN7vQE2Y54= # ← ツリーのルートハッシュ

— sum.golang.org Az3gri/oVBN1jAhNZ/iS+W31psFAWRT20/GyrhpU6taFwulQGetBkH5S7wpGXdS935jiIGiUUAXBVHzT/P6fYhqAmAk=
# ↑ sum.golang.org の署名
```

© 2019-2026 OPTiM Corp. All rights reserved.

STEP6 go.sum に記録する

OPTiM

- **検証を通ったハッシュをルールに従い go.sum に追記する**
  - ■**ビルド時にソースが必要とされるモジュール**はモジュール zip ファイルの h1 と /go.mod の h1 の**2行**
  - ■**モジュールグラフの構築にしか使わないモジュール**は /go.mod の h1 の**1行**
- **キャッシュは「探索でヒットしたモジュール全部」、go.sum は「ビルドに関与するモジュールだけ」という非対称がある**
  - ■github.com/go-chi/chi（ /v5 なし）v1.5.5 は、モジュール zip ファイルの取得・展開に加え checksum database での検証まで通る
  - ■しかし、 go.sum には 1 行も書かれない
  - ■理由は import path 解決のための「探り」であって、最終的なモジュールグラフの一員ではないため
- **つまり、 go.sum はダウンロードしたモジュールから手元で計算したハッシュと照らし合わせるための、正解のハッシュを書き留めたもの**
- **go mod verify はキャッシュの zip と展開済みディレクトリを再ハッシュし、ダウンロード時に書いた .ziphash と比較する**
  - ■STEP4 の再実行なので、キャッシュ汚染の確認に使える

© 2019-2026 OPTiM Corp. All rights reserved.

まとめ

© 2019-2026 OPTiM Corp. All rights reserved.

疑問への答え合わせ

OPTiM

問い	答え
なぜ同じディレクトリに配置しているのか？	同じ <module>@<version> なら中身の h1 ハッシュは 1つに決まる 1つ置けば、どのプロジェクトも同じものを使えるため
別のプロジェクトが別のバージョンを使いたくなったら？	どのバージョンを使うかは各プロジェクトの go.mod が決めるため、必要なバージョンを追加で置くだけで済む
別のプロジェクトが中身を書き換えたら？	読み取り専用なので書き込めない 無理に書き換えても go mod verify が go.sum との不一致でエラーになる
バージョンタグを別のコミットに付け替えたら？	go.sum と checksum database には最初のハッシュが残っている モジュールプロキシにキャッシュ済みなら、元の中身がそのまま配られる モジュールプロキシにキャッシュがないなら、付け替え後の中身はハッシュが合わずエラーになる
＋ α の疑問 checksum database の応答も偽のハッシュに書き換えたら？	checksum database の署名が合わないためエラーになる 公開鍵は Go コマンドの中にあるので、鍵ごと差し替えかえられない

© 2019-2026 OPTiM Corp. All rights reserved.

調べきれなかったもの

- 最後は Google を信用するしかないということなのか？
- Go 1.27.0 で依存管理がどう変わったのか？
- dep / Gopkg.lock の時代の依存管理の方法
- なぜ lock ファイルを辞めたのか？
- 標準パッケージが go.mod と go.sum に記録されない理由
  - ■どうやってインストールしているのか？
  - ■どうやって検証しているのか？

© 2019-2026 OPTiM Corp. All rights reserved.

OPTiM®

www.optim.co.jp

© 2019-2026 OPTiM Corp. All rights reserved.